

Linear Regression:

- Overfitting, under specified
- $\vec{w} = (X^T X)^{-1} X^T \vec{y}$ (RR: $(X^T X + \lambda I_n)^{-1} X^T \vec{y} = \vec{w}$)
- Primal form: $n > m$
 $\vec{w} = \mathcal{F}(\vec{x}) = \vec{w}^T \vec{x}$
- Dual form: $m > n$ (sparsity benefits)
 $\mathcal{F}(\vec{x}) = \sum_i \alpha_i \vec{x}_i^T \vec{x}$ ($\vec{w} = \sum_i \alpha_i \vec{x}_i$, $\alpha_i = \frac{1}{\lambda} (y_i - \vec{w}^T \vec{x}_i)$)
 $\vec{\alpha} = (X X^T + \lambda I_m)^{-1} \vec{y}$

Kernels:

- $k(x, t) = \langle \phi(x), \phi(t) \rangle$, $k(x, x) \geq 0$ (PSD function)

- Types: Polynomial

$$(1 + \langle x, t \rangle)^d$$

$$\phi(\vec{x}) = (1, \vec{x}, \vec{x}^2, \vec{x}^3, \dots)^T$$

Anova

$$\prod_i (1 + x_i t_i)$$

$$\phi(\vec{x}) = (1, x_1, x_2, \dots, x_n, x_1 x_2, \dots, x_1 x_n)^T$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

Gaussian

$$\exp[-\beta \|x - t\|^2]$$

$$\phi(x) = e^{-\beta \|x\|^2} \left(\sqrt{\frac{2\beta}{1!}} x, \sqrt{\frac{(2\beta)^2}{2!}} x^2, \dots \right)^T$$

Min

$$\min(x, t)$$

$$\phi(x) = \min(x, 0)$$

If function expressed as only dual inner products, can do kernel trick (by Dual form)

Online Learning:

Proof for Bounds

- Halving algorithm, $M \leq \log_2 n$ if consistent expert
- Weighted majority, $w_{t+1,i} \leftarrow w_{t,i} \beta^{|x_{t,i} - y_t|}$, $\beta \rightarrow 0$, $\beta \rightarrow 1$, $M \leq \log n$
- Perceptron, $\vec{w}_{t+1} \leftarrow \vec{w}_t + y_t \vec{x}_t$ on mistake
 $M \leq \left(\frac{R}{\gamma}\right)^2$, $R = \max_t \|\vec{x}_t\|$, $\gamma = \text{margin}$
 $(M, \gamma)^2 \leq \|\vec{w}_{m+1}\|^2 \leq M R^2$
induction (claim $\langle \vec{w}_t, \vec{v} \rangle \geq M, \gamma$)
- Winnow, $w_{t+1,i} \leftarrow w_{t,i} \times 2^{(y_t - g_i) \vec{x}_{t,i}}$
- DNF $\phi(\vec{x}) : \mathbb{B}^n \rightarrow \mathbb{B}^2$, ANOVA Kernel, $M \leq 3k(\log(n) + 1) + 2$

SVM:

- Lagrange multipliers, slack variables

Gaussian Processes:

- Prior, Noise, Posterior
- Error bars
- Evidence

$$P(\vec{w} | \vec{x}, y) \propto P(\vec{x}, y | \vec{w}) \cdot P(\vec{w})$$

$$\vec{w}_{\text{map}} = X^T (K + \sigma^2 I)^{-1} \vec{y}$$

$$\Sigma^{-1} = \frac{1}{\sigma^2} (X^T X + \sigma^2 I)$$

$$\log P(\vec{y} | S) = -\frac{1}{2} \vec{y}^T (K + \sigma^2 I) \vec{y} - \frac{1}{2} \log \det(K + \sigma^2 I) - \frac{n}{2} \log 2\pi$$

Trees & Ensembles:

- CART, grow & prune

- Weak learners, k

- Bagging & Random forest

- Boosting

- Training error $\frac{1}{m} \sum I(H(\vec{x}_i) \neq y_i) \leq \frac{1}{m} \sum e^{-y_i f(\vec{x}_i)} = \prod_t Z_t$

Learning Theory:

- $\text{err}(f) = E_{(\vec{x}, y) \in D} [V(f(\vec{x}), y)]$

- Generalisation

$$\text{err}(c) \leq \hat{\text{err}}(c) + \sqrt{\frac{\log C(H) + \log 1/\delta}{m}}$$

- Structural risk minimization

$$\text{err}(c^*) \leq \hat{\text{err}}(c^*) + \sqrt{\frac{C(H) + \ln 1/\delta + \ln K}{m}}$$

- $\text{err}(m, H, \delta) = \frac{2}{m} (d \ln \frac{2em}{d} + \ln 2/\delta)$

- Bias-Variance decomposition

$$\text{err}(f_S) - \text{err}(f^*) = \underbrace{\text{err}(f_S) - \text{err}(f_H)}_{\text{Variance}} + \underbrace{\text{err}(f_H) - \text{err}(f^*)}_{\text{Bias}}$$

COMPGIO1 - Supervised Learning

1: Introduction

• SL model:

- training set $S = (\vec{x}_i, y_i), i=1, \dots, m$ sampled iid from unknown $P(\vec{x}, y)$ & fixed

• Least Squares:

find f_0 s.t. $f_0(\vec{x}_i) \approx y_i$

$$\hat{y}_i = \vec{w}^T \vec{x}_i, \text{ minimise } E(\vec{w}) = \sum_i (\hat{y}_i - y_i)^2$$

$$E(\vec{w}) = \frac{1}{m} (\mathbf{X}\vec{w} - \vec{y})^T (\mathbf{X}\vec{w} - \vec{y})$$

$$\frac{\partial E}{\partial \vec{w}} = 0 \Rightarrow \mathbf{X}^T \mathbf{X} \vec{w} = \mathbf{X}^T \vec{y}$$

$\mathbf{X} = \begin{pmatrix} \vec{x}_1^T \\ \vdots \\ \vec{x}_m^T \end{pmatrix}, \vec{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix}$

- can add bias by substituting $\vec{x}^T = (\vec{x}^T, 1)$, $\vec{w}^T = (\vec{w}^T, b)$

• k-NN:

- $f(\vec{x})$ = class of majority of k nearest neighbours (by some dist metric, eg Euclidean)
- $\uparrow k$ = smoother
- asymptotically optimal:

$$P(Y=c|\vec{x}) = \frac{|\{i: y_i = c, i \in \{1, \dots, m\}\}|}{k}$$

Optimal Bayes estimator, minimizer of $E((y - f(\vec{x}))^2)$, w/ $\vec{x}, y$ sampled from $P(\vec{x}, y)$

as $m \rightarrow \infty$

$$E(k\text{-NN}) \rightarrow E(f^*) \text{ if } k(m) \rightarrow \infty, \frac{k(m)}{m} \rightarrow 0$$

(Convergence rate depends exponentially on x dimensionality)

• Square loss:

$$E(f) = \sum_{x,y} (y - f(x))^2 P(x,y) = \sum_x \left[\sum_y (y - f(x))^2 P(y|x) \right] P(x)$$

$$E(f(x')) \propto \sum_y (y - f(x'))^2 P(y|x')$$

$$\frac{\partial}{\partial f(x')} E(f(x')) = -2 \sum_y (y - f(x')) P(y|x') = 0$$

$$\Rightarrow f(x') = \sum_y y P(y|x')$$

$$\Rightarrow f^*(x) = \sum_y y P(y|x) = E(y|x) = F(x)$$

ie Bayes optimal estimator is

- Bias & Variance: $y = F(x) + \epsilon$, learner $A(x)$
 - underlying
 - noise

only for square loss!

$$E(A(x')) = E((y' - A(x'))^2) \quad [y' \text{ Sampled from } P(Y|x')]$$

$$= E(y' - f^*(x')^2) + \leftarrow \text{Bayes error (irreducible noise)}$$

$$E(f^*(x') - E(A(x')))^2 + \leftarrow \text{bias}^2 \text{ (discrepancy between algorithm \& truth)}$$

$$E((A(x') - E(A(x'))))^2 \leftarrow \text{variance (between training sets)}$$

- No Free Lunch:

usually no algorithm does almost as well as Bayes estimator

- Bayes classifier:

$$f^*(\vec{x}) = \arg\max_c P(Y=c|\vec{x})$$

$$\Rightarrow \text{Bayes error rate} = \int (1 - P(Y=f^*(\vec{x})|\vec{x})) dP(\vec{x})$$

- Hypothesis space:

approx $E(f)$ w/ $E_{\text{emp}}(S, f)$

- if f can be anything, can always find f s.t. $E_{\text{emp}}(S, f) = 0$
- need to limit to hypothesis space H , to reduce chance of overfitting

$$f_S = \arg\min_{f \in H} E_{\text{emp}}(S, f)$$

- Cross-validation:

method to approximate $E(f)$; useful to choose H /hyperparameters

$$S = [1 \mid 2 \mid \dots \mid k-1 \mid k]$$

- train on $k-1$ folds, test on held out fold
- repeat k times & average errors

$\uparrow k$ = better estimates
more expensive

1/2

2: Kernels & Regularization

• Ill posed problems:

- problem well posed is:
 - a solution exists
 - the solution is unique
 - solution depends continuously on data
- learning problems generally ill posed; regularization provides framework to solve these

• Ridge Regression

- add penalty term for more 'complex' hypothesis
- minimize $\epsilon_{\text{emp}, \lambda}(\vec{w}) = \sum_i (y_i - \vec{w}^T \vec{x}_i)^2 + \lambda \sum_i w_i^2$

$$\cdot \nabla \epsilon_{\text{emp}, \lambda}(\vec{w}) = 0 \Rightarrow -2X^T(\vec{y} - X\vec{w}) + 2\lambda\vec{w} = 0$$

$$\vec{w} = (X^T X + \lambda I_n)^{-1} X^T \vec{y}$$

• Dual Representation

$$\cdot \zeta(\vec{x}) = \vec{w}^T \vec{x}$$

↑
primal form

$$\vec{w} = \frac{1}{\lambda} (X^T(\vec{y} - X\vec{w}))$$

$$\vec{w} = \sum_{i=1}^m \alpha_i \vec{x}_i \quad (\alpha_i = \frac{1}{\lambda} [y_i - \vec{w}^T \vec{x}_i])$$

$$\text{dual form} \rightarrow \zeta(\vec{x}) = \sum_i \alpha_i \vec{x}_i^T \vec{x}$$

	Train time	Test time	Prediction time	Good when
Primal form	$O(mn^2 + n^3)$	$O(n)$		
Dual form	$O(nm^2 + m^3)$	$O(mn)$		
				$m \ll n$ ↑ ↑ # training points dimensionality of x

- if $\vec{x}$ is sparse (at most k non-zero components), $\vec{x}^T \vec{w}$ is $O(k)$

- if $km \ll n$, dual representation $O(km^2 + m^3)$ training & $O(mk)$ predicting
- if $k \ll m \ll n$

- Feature maps:

- Basis functions: explicit feature maps

$$\Phi: \mathbb{R}^n \rightarrow \mathbb{R}^N, \quad \vec{\Phi}(\vec{x}) = (\phi_1(\vec{x}), \dots, \phi_N(\vec{x}))^T, \quad x \in \mathbb{R}^n$$

- perform non-linear regression by transforming input into a higher dimensional space

primal representation $\rightarrow$

$$f(\vec{x}) = \sum_{j=1}^N w_j \phi_j(\vec{x}) = \vec{w}^T \vec{\Phi}(\vec{x})$$

- $m \ll N \Rightarrow$ better to work w/ dual representation

- Kernel functions: implicit feature map

- in dual form, don't need Φ explicitly; only need inner product between pairs of feature vectors

eg $k(x, t) = (x^T t)^2$

$\Phi(x) = (x_1^2, x_2^2,$

$x_1 x_2, \dots$

$x_2 x_1, \dots)$

$$K: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}, \quad K(\vec{x}, \vec{t}) = \langle \vec{\Phi}(\vec{x}), \vec{\Phi}(\vec{t}) \rangle, \quad \vec{x}, \vec{t} \in \mathbb{R}^n$$

$\Rightarrow$ computing $K(\vec{x}, \vec{t})$ independent of N !

- feature map of a given k is not unique

- $K: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ is positive semi-definite if symmetric & matrix $(K(x_i, x_j))_{i,j=1,\dots,K}$ is positive semi-definite for every $K \in \mathbb{N}$ & $x_1, \dots, x_K \in \mathbb{R}^n$

- K is PSD $\Leftrightarrow K(\vec{x}, \vec{t}) = \langle \vec{\Phi}(\vec{x}), \vec{\Phi}(\vec{t}) \rangle, \quad x, t \in \mathbb{R}^n$
for some $\vec{\Phi}: \mathbb{R}^n \rightarrow W$ & Hilbert space W

- Example kernels:

- Polynomial $K(x, t) = (1 + x^T t)^d$

- Arava $K(x, t) = \prod (1 + x_i t_i)$

- Gaussian $K(x, t) = \exp[-\beta \|x - t\|^2]$

- min $K(x, t) = \min(x, t) \quad (x, t \in [0, \infty])$

∞ dimensional
corresponding
feature map!

2/2

2: kernels & Regularization

• Kernel construction:

• k_1, k_2 are kernels, & $\alpha > 0$, A is PSD & symmetric, k is kernel on $\mathbb{R}^N$ & $\phi: \mathbb{R}^N \rightarrow \mathbb{R}^N$, the following are PSD kernels on $\mathbb{R}^N$:

$$\begin{aligned} & \vec{x}^T A \vec{f} \\ & k_1(\vec{x}, \vec{f}) + k_2(\vec{x}, \vec{f}) \\ & \alpha k_1(\vec{x}, \vec{f}) \end{aligned} \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{-- so can have polynomial} \\ & k_1(\vec{x}, \vec{f}) k_2(\vec{x}, \vec{f}) \quad \text{of kernels too} \\ & k(\phi(\vec{x}), \phi(\vec{f}))$$

• Dual Representation:

• Representer theorem: dual form of RR holds for other loss functions too

$$\mathcal{E}_{\text{emp}, \lambda}(\vec{w}) = \sum_i V(y_i, \langle \vec{w}, \vec{\phi}(\vec{x}_i) \rangle) + \lambda \langle \vec{w}, \vec{w} \rangle, \quad \lambda > 0$$

where $V: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is a loss function

$$\nabla \mathcal{E}_{\text{emp}, \lambda}(\vec{w}) = 0 \Rightarrow \sum_i V'(y_i, \langle \vec{w}, \vec{\phi}(\vec{x}_i) \rangle) \vec{\phi}(\vec{x}_i) = 2\lambda \vec{w}$$

$$\Rightarrow \vec{w} = \sum_i \alpha_i \vec{\phi}(\vec{x}_i) \quad (\alpha_i = \frac{1}{2\lambda} V'(y_i, \langle \vec{w}, \vec{\phi}(\vec{x}_i) \rangle))$$

$$\Rightarrow g(\vec{x}) = \langle \vec{w}, \phi(\vec{x}) \rangle$$

$$= \sum_i \alpha_i \langle \vec{\phi}(\vec{x}_i), \vec{\phi}(\vec{x}) \rangle$$

$$\underline{g(\vec{x}) = \sum_i \alpha_i k(\vec{x}, \vec{x}_i)}$$

kernel
trick

1/2

3: Online Learning

- Model: online sequence of data (usually no distributional assumptions)
 - sequentially predict & update classifier, aiming to minimize cumulative loss
 - + fast, low memory, no stat assumptions
 - non-asymptotic, no stat assumptions
 - Learning with Expert Advice:
 - each expert makes a prediction, goal is to combine predictions $\vec{x}_t$ to predict $\hat{y}_t$, estimate of y_t
 - Halving algorithm:
 - every time step, predict w/ majority of remaining experts
 - remove incorrect experts
 - on mistake, at least half experts removed; $\leq \log_2 n$ mistakes (if a perfect expert exists)
 - if no consistent expert, $L_A(S) \leq \alpha \cdot \min_i L_i(S) + \beta \log n$
 α, β constants
 $L_i(S)$ loss of i^{th} expert
 - Weighted majority algorithm:
 - every expert has a weight
 - predict w/ weighted majority
 - wrong expert weights multiplied by $\beta \in [0, 1)$
 - Mistakes bound:

$$W_{\text{minority}} \leq \frac{1}{2} W_t, \quad W_{\text{majority}} \geq \frac{1}{2} W_t$$
 - if no mistake, minority $\times \beta$,

$$W_{t+1} \leq \beta W_t$$
 - if mistake, majority $\times \beta$,

$$W_{t+1} \leq (1 + \beta) \frac{1}{2} W_t$$
- $$\Rightarrow W_{m+1} \leq \left(\frac{1+\beta}{2}\right)^m W_1 = \sum_{\text{experts}} W_{m+1}^{\text{expert}} = \sum_{\text{experts}} \beta^{M_{\text{expert}}} \geq \beta^{M_i}$$
- $$\Rightarrow \left(\frac{1+\beta}{2}\right)^m W_1 \geq \beta^{M_i} \Rightarrow M \leq \alpha \cdot \min_i M_i + \beta b \cdot \log n$$
- ie comparable to best expert

- Learning with thresholded linear combinations
 - algorithm predicts $\hat{y}_t$ w/ using $\vec{x}_t$ & past experience, & loss $L_A(S)$ on whole sequence
 - compare performance w/ $\mathcal{U} = \{\vec{u}\}$, ie hypothesis space, here $\vec{u}$ is a linear threshold function
 - assume $\exists \vec{u}_0, \text{Loss}_{\vec{u}_0}(S) = 0$

linear threshold function $\rightarrow \mathcal{F}_{\vec{u}, b}(\vec{x}) = \text{sign}(\vec{u} \cdot \vec{x} + b)$

Variables / experts $\rightarrow$

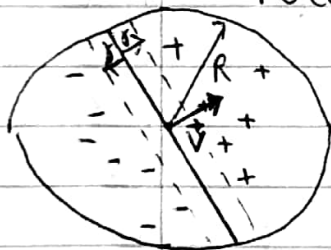
E_1	E_2	E_3	E_4	y	E_1, E_2	E_1, E_3	$\dots$
1	0	1	0	1	1	1	
0	1	0	0	0	1	0	

2 mistakes 3 mistakes

Naive solution:

Weighted Majority over all possible disjunctions
 n literals $\rightarrow \binom{n}{k}$ weights; exponential
 k on each disjunction in time & space!

Perceptron:



assume data linearly separable by margin δ , s.t. hyperplane exists w/ normal $\vec{v}$ where $\|\vec{v}\| = 1$

$$\forall t, y_t \in \{-1, 1\}, \forall \vec{x}_t, \|\vec{x}_t\| \leq R, \\ \forall (\vec{x}_t, y_t) \cdot y_t (\vec{x}_t \cdot \vec{v}) \geq \delta$$

$\vec{w}_1 = \vec{0} \quad M_1 = 0$

For $t = 1 \dots m$:

$\vec{x}_t \in \mathbb{R}^n$

$\hat{y}_t = \text{sign}(\vec{w}_t \cdot \vec{x}_t)$

if $\hat{y}_t \neq y_t$:

$\vec{w}_{t+1} = \vec{w}_t + y_t \vec{x}_t, \quad M_{t+1} = M_t + 1$

else:

$\vec{w}_{t+1} = \vec{w}_t, \quad M_{t+1} = M_t$

Mistakes: $M \leq \left(\frac{R}{\delta}\right)^2 \quad R = \max_t \|\vec{x}_t\|$

(note $\vec{w}_{m+1}$ does not necessarily linearly separate data set S ,
 as M does not depend on m , the size of S)

2/2

3: Online Learning

Perceptron mistake bound on disjunction learning:

$$M \leq R^2 \|\vec{u}\|^2 \quad (\|\vec{u}\|^2 = \frac{1}{\gamma^2})$$

$\vec{x} \in \{0,1\}^n$, use feature map $\phi(\vec{x}) = (\vec{x}, 1)$

$\vec{u}^* \in \mathbb{R}^{n+1}$ separates w/ margin of 1:

$$u_i^* = \begin{cases} 2 & i \text{ is a literal} \\ 0 & i \text{ is not a literal} \\ -1 & i \text{ is bias} \end{cases}$$

no. literals



$$\Rightarrow \|\vec{u}\|^2 = 4k+1, \quad \|\phi(\vec{x})\|^2 \leq (n+1) \Rightarrow \underline{M \leq (4k+1)(n+1)}$$

Winnow:

Algorithm:

$$\vec{w}_1 = \vec{1}$$

for $t=1 \dots m$:

$$\hat{y}_t = \begin{cases} 0 & \text{if } \vec{w}_t \cdot \vec{x}_t < n \\ 1 & \text{otherwise} \end{cases}$$

$$\text{if } \hat{y}_t \neq y_t \text{ then } w_{t+1,i} = w_{t,i} \times 2^{(y_t - \hat{y}_t)x_{t,i}}$$

Mistakes:

$$W_t = \sum_i w_{t,i}, \quad W_1 = n$$

positive mistake

→ on mistake, $y_t = 1$, $W_{t+1} \leq W_t + n$

negative mistake → on mistake, $y_t = 0$, $W_{t+1} \leq W_t - \frac{n}{2}$

$$\Rightarrow W_{m+1} \leq n + M_p n - M_g \frac{n}{2}$$

$$\Rightarrow M_g \leq 2k(\log n + 1) + 2$$

$$\Rightarrow \underline{M \leq 2 + 3k(\log n + 1)}$$

$M_p \leq k(\log n + 1)$
as on positive mistake relevant weight doubles, & when $w_{t,i} \geq n$ it will definitely no longer change

exponential improvement over Perceptron in n !

reduce α_n $\phi(a) \cdot \phi(b)$
to $O(1)$
 $k(a, b)$

$$\phi(\vec{x}) = (\vec{x}, 1)$$

ANOVA
Kernel

• DNF case study:

$$x_1 x_2 x_3 \vee \bar{x}_1 \bar{x}_3 \vee x_2 x_3$$

$$w_t = \sum_{q \in \text{features}} \alpha_q \cdot \phi(\vec{x}_q)$$

$$\vec{w}_t \cdot \phi(\vec{x}_t) = \sum_q \alpha_q \cdot \phi(\vec{x}_q) \phi(\vec{x}_t) = \sum_q \alpha_q k(\vec{x}_q, \vec{x}_t)$$

• Perceptron:

• Prediction: $O(n \times \# \text{mistakes})$

• Mistake bound: $O(k \cdot 2^n)$

→ fast, poor bound

• Winnow:

• Prediction

$O(2^n \times \# \text{mistakes})$

→ slow, good bound

• Mistake bound

$O(k \cdot \ln 2^n)$

$$w_{t,i} = \exp[-\eta \sum_q \alpha_q \phi(\vec{x}_q)_i]$$

cannot do kernel trick

5: Support Vector Machines

- Goal: given data $S = \{(\vec{x}_i, y_i)\}_{i=1}^m \in \mathbb{R}^d \times \{-1, 1\}$
find a maximally separating hyperplane, ie

$$H_{\vec{w}, b} = \{\vec{x} \in \mathbb{R}^d : \vec{w}^T \vec{x} + b = 0\}$$

$$\text{(classify } \hat{y} = \begin{cases} 1 & \text{if } \vec{w}^T \vec{x} + b \geq 0 \\ -1 & \text{if } \vec{w}^T \vec{x} + b < 0 \end{cases})$$

dist of $\vec{x}_i$ from $H_{\vec{w}, b}$ $\rightarrow \rho_{\vec{x}}(\vec{w}, b) = \frac{|\vec{w}^T \vec{x} + b|}{\|\vec{w}\|} = \frac{y_i(\vec{w}^T \vec{x} + b)}{\|\vec{w}\|}$ margin of $\vec{x}_i$

margin of separating hyperplane $\rightarrow \rho_s(\vec{w}, b) = \min_i \rho_{\vec{x}_i}(\vec{w}, b)$

if hyperplane separates data
(-ve for incorrect y_i if not)

We want to maximise

$$\rho(S) = \max_{\vec{w}, b} \left[\min_i \left(\frac{y_i(\vec{w}^T \vec{x}_i + b)}{\|\vec{w}\|} \cdot y_i(\vec{w}^T \vec{x}_i + b) \geq 0, i=1, \dots, m \right) \right]$$

canonical hyperplane $\rightarrow$ cannot solve, no unique solution; need to restrict $\|\vec{w}\|$

• set $\|\vec{w}\| = \frac{1}{\rho_s(\vec{w}, b)}$ (so $\min_i [y_i(\vec{w}^T \vec{x}_i + b)] = 1$)

$$\rho(S) = \max_{\vec{w}, b} \min_i [y_i(\vec{w}^T \vec{x}_i + b)] \rightarrow \text{merge to } \dots \geq 1$$

$$\rho(S) = \max_{\vec{w}, b} \left[\frac{1}{\|\vec{w}\|} \cdot \min_i (y_i(\vec{w}^T \vec{x}_i + b)) = 1, y_i(\vec{w}^T \vec{x}_i + b) \geq 0 \right]$$

$$\rho(S) = \min_{\vec{w}, b} \frac{1}{\min_i [y_i(\vec{w}^T \vec{x}_i + b) \geq 1]}$$

- So want to minimize $\frac{1}{2} \vec{w}^T \vec{w}$, subject to $y_i (\vec{w}^T \vec{x}_i + b) \geq 1, \forall i$
- Equivalent to finding saddle point of Lagrangian function

$$L(\vec{w}, b; \alpha) = \frac{1}{2} \vec{w}^T \vec{w} - \sum_{i=1}^m \alpha_i (y_i (\vec{w}^T \vec{x}_i + b) - 1) \quad (1)$$

($\alpha_i \geq 0$ are Lagrange multipliers)

$$\frac{\partial L}{\partial b} = -\sum_i y_i \alpha_i = 0$$

$$\frac{\partial L}{\partial \vec{w}} = \vec{w} - \sum_i \alpha_i y_i \vec{x}_i = 0 \Rightarrow \vec{w} = \sum_{i=1}^m \alpha_i y_i \vec{x}_i \quad (2)$$

linear combination of $\vec{x}_i$
($\vec{x}_i$ called Support Vectors, or SVs)
SVs usually small subset of data

• Dual problem:

- sub (2) into (1) to get Dual problem; now complexity scales w/ m , not d

• Linearly non-separable case:

- Introduce slack variables ϵ_i to allow some points to have margin < 1
 - Minimise $\frac{1}{2} \vec{w}^T \vec{w} + C \sum \epsilon_i$
subject to $y_i (\vec{w}^T \vec{x}_i + b) \geq 1 - \epsilon_i$
 $\epsilon_i \geq 0, i = 1 \dots M$
- C controls trade off between $\|\vec{w}\|^2$ & training error, $\sum \epsilon_i$

$$\text{again, } \vec{w} = \sum \alpha_i y_i \vec{x}_i,$$

- C is equivalent to regularisation (ridge regression style, w/ $\lambda = \frac{1}{2C}$)

• Kernels:

- can use a feature map $\phi(\vec{x})$, and since use the kernel trick, so

$$g(\vec{x}) = \sum y_i \alpha_i K(\vec{x}_i, \vec{x}) + b \quad (\text{classify with } \text{sign}(g(\vec{x})))$$

• Regression:

$$\text{loss} = |y - g(\vec{x})|_\epsilon = \max(|y - g(\vec{x})| - \epsilon, 0)$$

(errors below ϵ don't count $\Rightarrow$ sparse solutions!)

G: Gaussian Processes

• Bayesian inference:

equivalent to 2-norm regularization

• Prior: distribution over possible hypothesis
 $\rightarrow \mathcal{F} = \{f(\vec{x}) = \langle \vec{w}, \phi(\vec{x}) \rangle : \vec{w} \in \mathcal{F}\}$, and $\vec{w} \sim N(\vec{0}, I)$

• Additive noise model:

• measurements of outputs corrupted by additive noise

equivalent to minimising squared discrepancy; Least Squares Regression

$\rightarrow y_i | \vec{w}, \vec{x}_i \sim N(\langle \vec{w}, \phi(\vec{x}_i) \rangle, \sigma^2)$

output of function is 'true' output, then corrupted by additive noise

• Posterior:

$$dP_{\text{post}}(\vec{f}) = \underset{\text{prior}}{dP(\vec{f})} \cdot \underset{\text{likelihood (of learning data } S_x)}{P(\vec{y} | S_x, \vec{f})}$$

$$\Rightarrow dP_{\text{post}}(\vec{w}) \propto \exp\left[-\frac{\|\vec{w}\|^2}{2}\right] \cdot \exp\left[\frac{-\sum (y_i - \langle \vec{w}, \phi(\vec{x}_i) \rangle)^2}{2\sigma^2}\right]$$

• MAP inference: max of posterior

• as exp is monotonic,

$$\vec{w}_{\text{MAP}} = \underset{\vec{w}}{\text{argmin}} \sum (y_i - \langle \vec{w}, \phi(\vec{x}_i) \rangle)^2 + \sigma^2 \|\vec{w}\|^2$$

• equivalent to Ridge Regression problem, w/ $\lambda = \sigma^2$

$$\Rightarrow \vec{w}_{\text{MAP}} = X^T (K + \sigma^2 I)^{-1} \vec{y} \quad (\text{where } K \text{ is the kernel matrix})$$

• Dual solution:

$$\begin{aligned} \text{sub in } \vec{w}_{\text{MAP}} = X^T \alpha &\Rightarrow \sigma^2 \alpha = \vec{y} - K \alpha \\ &\Rightarrow (K + \sigma^2 I_m) \alpha = \vec{y} \end{aligned}$$

$$\Rightarrow \alpha = (X X^T + \sigma^2 I_m)^{-1} \vec{y}$$

$$\text{and } g(\vec{x}) = \vec{x}^T \vec{w} = \vec{x}^T X^T \alpha = \sum_{i=1}^m \alpha_i k(\vec{x}, \vec{x}_i)$$

regression function

prior $\rightarrow P(\vec{z}|S) = \frac{1}{\sqrt{\det(2\pi K)}} \exp\left[-\frac{1}{2} \vec{z}^T K \vec{z}\right]$

noise model $\rightarrow P(\vec{y}|\vec{z}) = \frac{1}{(2\pi\sigma^2)^{m/2}} \exp\left[-\frac{1}{2\sigma^2} \|\vec{y} - \vec{z}\|^2\right]$

• Gaussian Process Regression:

• use framework for Bayesian inference

• Prior: for $S = \{\vec{x}_1, \dots, \vec{x}_m\}$ training examples,
 $y_1, \dots, y_m$ corresponding outputs
 distributed by multidimensional Gaussian w/
 mean $\vec{0}$ & covariance q , kernel matrix

• a point in this represents an assignment of all inputs to a specific output

• Inference:

$$p_{\text{post}}(\vec{w}|S) \propto \exp\left[-\frac{\sum (y_i - \langle \vec{w}, \phi(\vec{x}_i) \rangle)^2}{2\sigma^2} - \frac{\|\vec{w}\|^2}{2}\right]$$

$$\propto \exp\left[-\frac{1}{2} (\vec{w} - \vec{w}_{\text{map}})^T \Sigma^{-1} (\vec{w} - \vec{w}_{\text{map}})\right]$$

$$\Rightarrow \vec{w}|S \sim N(\vec{w}_{\text{map}}, \Sigma)$$

$$\Sigma^{-1} = \frac{1}{\sigma^2} (X^T X + \sigma^2 I)$$

• Error bars:

$$y = \vec{w}_{\text{map}}^T \phi(\vec{x}) + \vec{v}^T \sqrt{\Sigma} \phi(\vec{x}), \text{ where } \vec{v} \sim N(\vec{0}, I)$$

$$\begin{aligned} \sigma_y^2 &= E[(y - \vec{w}_{\text{map}}^T \phi(\vec{x}))^2] = \phi(\vec{x})^T \sqrt{\Sigma} \vec{v} \vec{v}^T \sqrt{\Sigma} \phi(\vec{x}) \\ &= \phi(\vec{x})^T \Sigma \phi(\vec{x}) \\ &= k(\vec{x}, \vec{x}) - k^T \end{aligned}$$

• Evidence:

• probability of the data

$$P(\vec{y}) = \int_S p_{\text{post}}(\vec{g}|S) d\vec{g}$$

• for a GP:

$$P(\vec{y}|S) = \int P(\vec{y}|\vec{z}) P(\vec{z}|S) d\vec{z}$$

$$\rightarrow \log(P(\vec{y}|S)) = -\frac{1}{2} \vec{y}^T (K + \sigma^2 I)^{-1} \vec{y} - \frac{1}{2} \log \det(K + \sigma^2 I)$$

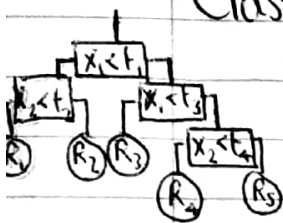
$$-\frac{m}{2} \log 2\pi$$

• can be good for model selection, over different σ or kernel hyperparameters

7: Tree based learning & Ensemble Methods

Classification & Regression trees:

- Divide space into hyperrectangles by recursive binary partition



Regression:

- $C_n = \text{ave}(y_i | \vec{x}_i \in R_n)$, i.e. minimize square error in region n

- goal: $\min_{R_1, \dots, R_N} \left[\sum_{i=1}^m \left(y_i - \sum_{n=1}^N \text{ave}(y_j | \vec{x}_j \in R_n) I[\vec{x}_i \in R_n] \right)^2 \right]$

- but computationally intractable; need to use heuristic

• Greedily grow tree:

- Search for best split to minimize square error by splitting data in 2

• $O(dm)$, as split can be in any dimension & be between any 2 training points

- repeat until max no. data at each node (eg 5)

• Cost-complexity pruning:

- this tree will overfit training data; need to find $T_\lambda \subseteq \hat{T}$ minimizing

$$C_\lambda(T) = \sum_{n=1}^N m_n Q_n(T) + \lambda |T|$$

n goes over all leaves in T

m_n is no. points in that leaf

$$Q_n(T) = \frac{1}{m_n} \sum_{\vec{x}_i \in R_n} (y_i - C_n)^2$$

$$\Rightarrow \sum_{n=1}^N m_n Q_n(T) = \text{training error}$$

• algorithm: weakest link pruning:

- successively collapse nodes giving smallest per node increase in training error
- T_λ is in this sequence of subtrees

• Classification:

- Classify by majority vote in region

• Growing tree:

- minimise one of:

- Gini index $= \sum_k p_{nk} (1 - p_{nk})$

- Cross entropy $= \sum_k p_{nk} \log \frac{1}{p_{nk}}$

• Pruning tree:

- prune smallest misclassification error

$$= 1 - p_{nR(n)}, \quad R(n) = \underset{k}{\operatorname{argmax}} p_{nk}$$

empirical class
probs. for region n

$$p_{nR} = \frac{1}{m_n} \sum_{\vec{x} \in R_n} \mathbb{I}(y_i = k)$$

• Weak learner:

- algorithm which consistently finds classifiers slightly better than random

• Ensemble methods:

- Wisdom of Crowds
- Single prediction may be wrong, but crowd majority often can be correct
- eg, h_i predicts correct w/ probability $1/2 + \gamma$ (independently)

$$H_T = \operatorname{Sign} \left(\sum_{i=1}^{2T+1} h_i \right)$$

$$P(H_T \text{ wrong}) = \sum_{i=0}^T \binom{2T+1}{i} \left(\frac{1}{2} + \gamma \right)^i \left(\frac{1}{2} - \gamma \right)^{2T+1-i}$$

by Chernoff bound, $\leq \exp\left(-\frac{4}{5} \gamma^2 T\right) \leftarrow \text{exponential decay w/ } T \text{ or } \gamma^2!$

• Bagging:

- reduce classifier variance by training T variations, each on M examples (sampled with replacement), $\Rightarrow \sim 63\%$ ~~dist~~ non-repeats
- conventionally, $M = \text{size of training set}$

• Random forests:

- Decision trees high variance; use bagging with these
- to make further decorrelated, only use a subset of k features for each tree
- k usually $= \sqrt{d}$ or $\log d$

7: Tree based learning & Ensemble Methods

• Boosting: (Adaboost)

• Initialise $D_1(1) \dots D_1(m) = \frac{1}{m}$ (weights on training points)

• for $t = 1 \dots T$:

• fit $h_t: \mathbb{R}^d \rightarrow \{-1, 1\}$ using D_t ^{weighted training error}

• Choose α_t ($\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}$, $\epsilon_t = \sum_i D_t(i) I(h_t(\vec{x}_i) \neq y_i)$)

must keep $\sum D_t(i) = 1$ $\rightarrow D_{t+1}(i) \propto D_t(i) \cdot \exp[-\alpha_t y_i h_t(\vec{x}_i)]$ $\leftarrow$ 'harder' examples get higher weight

So need to divide all by Z_t $\rightarrow$ return $H(\vec{x}) = \text{sign}\left(\sum_t \alpha_t h_t(\vec{x})\right)$ $\leftarrow$ linear combination of weak learners, weights controlled by training errors

• Convergence:

• training error: $\frac{1}{m} \sum_i I(H(\vec{x}_i) \neq y_i) \leq \frac{1}{m} \sum_i e^{-y_i f(\vec{x}_i)} = \prod_t Z_t$

① as $H_i(\vec{x}) \neq y_i \Rightarrow e^{-y_i f(\vec{x}_i)} \geq 1$

② as $D_{t+1}(i) = \frac{1}{m} \cdot \frac{1}{Z_t} \cdot \prod_t e^{-\alpha_t y_i h_t(\vec{x}_i)}$ (& expand $\sum e^{-y_i f(\vec{x}_i)}$)

$\Rightarrow$ need to try to minimise Z_t every step

$$Z_t = \sum_i D_t(i) \exp[-\alpha_t y_i h_t(\vec{x}_i)] = \epsilon_t e^{\alpha_t} + (1 - \epsilon_t) e^{-\alpha_t}$$

$\Rightarrow$ minimised by $\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}$

$$\Rightarrow Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)} = \sqrt{1 - 4\delta_t^2}, \quad \delta_t = \frac{1}{2} - \epsilon_t$$

$$\Rightarrow \text{Train error} \leq e^{-2 \sum_t \delta_t^2} \approx e^{-2T\delta^2} \text{ if } \delta_t^2 \geq \delta^2 \geq 0$$

• Additive models:

• AdaBoosting greedily solves

$$\min \left[\sum_i V(y_i, \sum_t \alpha_t h_t(\vec{x}_i)) \right], \text{ where } V(y, \hat{y}) = e^{-y\hat{y}}$$

exponential loss

also, for fixed α_t , minimising exp loss for h_t equiv to minimising its misclassification error

Exponential loss as it punishes negative margins, & slightly promotes larger positive margins (whilst square loss would punish larger positive margins; bad!)

Summary:

- CART good as interpretable & work on ordinal data; less feature cleaning needed
- Bagging used on full trees, reduce variance
- Boosting often only use single split stumps, reduces bias
 - boosting often struggles w/ noisy data

9: Learning Theory

- $S = \{(\vec{x}_1, y_1), \dots, (\vec{x}_m, y_m)\}$, iid sample from unknown $P(\vec{x}, y)$

Unknown,
true
error

$$\rightarrow \epsilon(f) = E[V(y, f(\vec{x}))] = \int V(y, f(\vec{x})) dP(\vec{x}, y) = C_0$$

empirical
error
= training error

$$\rightarrow \epsilon_{\text{emp}}(f) = \frac{1}{m} \sum_i V(y_i, f(\vec{x}_i)) = \hat{C}_S$$

- Overfitting:

- minimising ϵ_{emp} leads to overfitting if model complexity unlimited
- we want to estimate true error (test error) & minimize this

- Test set bound? ($d = \text{dimensions of } \vec{x}$)

$$C_0 \leq \hat{C}_S + \sqrt{\frac{\ln(1/\delta)}{2m}} \quad \text{if } S \text{ is iid samples, with probability } 1 - \delta \text{ (for } \delta > 0)$$

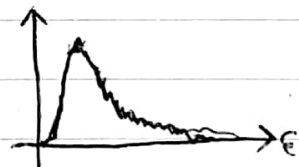
($m = \text{size of } S$)

- Generalisation of a learner:

- algorithm A chooses function $A_S(S)$ from function space $\mathcal{F}$, in response to training set S

random
variable

$$\rightarrow \epsilon(S, A, \mathcal{F}) = E_{(\vec{x}, y)} [V(A_S(S), \vec{x}, y)]$$



- can be approximated w/ cross-validation (get distribution of this RV)
- expected value of this RV can be misleading; we should focus on confidence bounds ($\delta = \text{confidence parameter}$)

Probably
Approximately
Correct

- Source of PAC; δ chance we've been 'misled' by training set

- Prob of being misled in classification:

$$P_S(\text{err}_S(f) = 0, \text{err}(f) > \epsilon) = (1 - \text{err}(f))^m \leq (1 - \epsilon)^m \leq e^{-\epsilon m}$$

$$\Rightarrow \epsilon = \ln(1/\delta) / m \quad \text{bounds probability less than } \delta$$

- Countable function classes:

$$\mathcal{F} = \{f_1, \dots, f_n, \dots\}, \quad \text{let prob of being misled by } f_n < q_n, \text{ where } \sum_n q_n \leq 1$$

$$\Rightarrow q_n \delta = P\left(\bigcap_{n=1}^{\infty} A_n\right), A_n = \{\text{err}_S(f_n) = 0, \text{err}(f_n) > \frac{1}{n} \ln \frac{1}{q_n \delta}\}$$

$$\sum_n P(A_n)$$

$$\Rightarrow P_S\{\exists f_n, \text{err}_S(f_n) = 0, \text{err}(f_n) > \frac{1}{n} \ln \frac{1}{q_n \delta}\} \leq \delta$$

$$\Rightarrow \text{err}(f_n) \leq \frac{1}{n} \left[\ln \frac{1}{q_n} + \ln \frac{1}{\delta} \right] \quad (\text{if } \text{err}_S(f_n) = 0)$$

note, must be prior weights to functions; expected generalisation minimal if weights = q_n

By Hoeffding's inequality, also have

$$E(f_S) \leq E_{\text{emp}}(f_S) + \sqrt{\log 1/d / 2m}$$

let $f_S = \arg \min_{f_S} E_{\text{emp}}(f_S)$, with corresponding q_S

$$E(f_S) \leq E_{\text{emp}}(f_S) + \sqrt{\frac{\log 1/q_S + \log 1/\delta}{2m}} \quad \text{with confidence } 1-\delta$$

Sample complexity bound:

finite H , $q_n = 1/|H|$

samples to
avoid overfitting
w/ confidence $1-\delta$

$$\Rightarrow m(\epsilon, H, \delta) = \frac{\log |H| + \log 1/\delta}{2\epsilon^2}$$

Structural risk minimisation:

model selection approach to choosing hypothesis space from $H_1, \dots, H_k$

let $f_{S,q}$ be minimizer of E_{emp} in H_q

$$E(f_{S,q}) \leq E_{\text{emp}}(f_{S,q}) + \sqrt{\frac{\log |H_q| + \log 1/\delta}{2m}} \quad \text{w/ confidence at least } 1-\delta$$

we choose H_q to minimise rhs of this

9: Learning Theory

• Uncountable H

- need to convert to finite
- replace test point w/ test set, i.e. a second 'ghost' sample
- Double sample trick:

$$A(h) = \{\text{err}_x(h) = 0\}, B(h) = \{\text{err}(h) \geq \epsilon\}, C(h) = \{\text{err}_y(h) \geq \frac{\epsilon}{2}\}$$

$$P^{2m}(C(h) | A(h), B(h)) = P^{2m}(C(h) | B(h)) > 0.5$$

$$\Rightarrow P^{2m}(XY \in X^{2m} : \exists h \in H : A(h), C(h)) \geq P^{2m}(\text{ " " " } : A(h), B(h), C(h)) = P^{2m}(\text{ " " " } : A(h), B(h)) \cdot P(C(h) | A(h), B(h))$$

$$\Rightarrow P^m(X \in X^m : \exists h \in H : A(h), B(h))$$

$$\wedge 2P^{2m}(XY \in X^{2m} : \exists h \in H : A(h), C(h))$$

$$\wedge 2B_H(2m) P^{2m}(XY \in X^{2m} : \exists h \in H : A(h), C(h))$$

$$\left[\text{where } B_H(m) \leq \sum_{i=0}^d \binom{m}{i} \leq \left(\frac{em}{d}\right)^d, \text{ where } d = \text{VCdim}(H) \right]$$

max cardinality of
set of functions H when restricted
to m points

• Finally, by symmetrisation,

$$P^{2m}[XY \in X^{2m} : A(h), C(h)] \leq E^{2m}[P_{\sigma \sim \Sigma}[A(h), C(h) \text{ for } \sigma(XY)]]$$

$$\Rightarrow \boxed{\epsilon(m, H, \delta) = \frac{2}{m} \left(d \log \frac{2em}{d} + \log \frac{2}{\delta} \right)} \quad 2^{-\epsilon m/2} = E^{2m}(2^{-\epsilon m/2})$$

bound on
generalisation error
w/ confidence $1-\delta$

VC-dimension
(complexity/capacity)
of H

• Similar for non-zero training error; introduces a square root & significantly worse

• Bias/Variance decomposition:

$$\mathcal{E}(f_S) - \mathcal{E}(f_H^*) = \overbrace{\mathcal{E}(f_S) - \mathcal{E}(f_H)}^{\text{variance}} + \overbrace{\mathcal{E}(f_H) - \mathcal{E}(f_H^*)}^{\text{bias}}$$

$f_H = \operatorname{argmin}_{f \in H} \mathcal{E}(f)$

$$\begin{aligned} \mathcal{E}(f_S) - \mathcal{E}(f_H) &\leq \mathcal{E}(f_S) - \mathcal{E}_{\text{emp}}(f_S) + \mathcal{E}_{\text{emp}}(f_H) - \mathcal{E}(f_H) \\ &\leq 4 \sqrt{2 \frac{h \log(2^m/h + 1) + \log^2 1/\delta}{m}} \end{aligned}$$

- variance $\uparrow$ w/ complexity of H
 $\downarrow$ w/ sample size m
- bias $\downarrow$ w/ complexity of H
independent of m

• Criticisms of PAC theory:

- doesn't agree w/ practice, eg SVM
 - infinite dimensional feature space $\Rightarrow$ infinite VC dimension
 - but works well in practice

10: Sparsity Methods

• Boosting Generalisation error:

- usually decreases even after train error = 0
 $\Rightarrow$ Margin ~~increasing~~ _{distribution} still keeps decreasing

$$\text{Margin}_i = y_i f(\vec{x}_i)$$

$$\text{Generalisation Error} \leq P_{\text{emp}}(\text{margin} < \theta) + O\left(\frac{\sqrt{d/m}}{\theta}\right)$$

(d is VC dimension of set of weak learners)

• L_1 -Sparsity:

- Rademacher complexity alternative measure
- not increased by taking convex close of $\mathcal{H}$